Swift Cheat Sheet

for TypeScript developers · arrivals-board

VARIABLES & BASIC TYPES

Swift	TypeScript
`let x = 3`	`const x = 3` (can't reassign)
`var x = 3`	`let x = 3` (can reassign)

```
let count: Int = 84          // Int, Double, String, Bool
let lat = 37.6377            // type inferred: Double
Double(count) + lat          // no auto Int/Double mix
Double("37.6"), Int("12")    // text to number: optional
```

String interpolation: `"\(name) in \(mins) min"` (TS `${ }`)

OPTIONALS: `T?` = VALUE OR `NIL`

A box that holds a value or is empty. You must unwrap it before use.

```
let maybe: String? = dict["SBRN"]

if let name = maybe { print(name) } // only in braces
if let maybe { print(maybe) }   // shorthand

guard let name = maybe else { exit(1) } // must exit
print(name)                      // usable after guard

maybe ?? "Unknown"     // fallback, result not optional
maybe?.count           // chaining, still optional
maybe!                 // force unwrap: CRASHES if nil
```

- `guard` exits: `return` (in func), `continue` / `break` (in loop), `exit(1)` / `fatalError("...")` (top-level main.swift)
- Several at once: `guard let a = x, let b = y else { return }`
- **Trap:** `if let n: String? = maybe` always succeeds. Annotate the *unwrapped* type (no `?`).
- `guard let` / `if let` only work on optionals.
- Printed `Optional("...")`? You forgot to unwrap.

ARRAYS, TUPLES, DICTIONARIES

```
var ids: [String] = ["W30-1", "W30-2"]   // TS string[]
ids[0]  ids.count  ids.first  ids.last  ids.append("X")
ids.contains("W30-1")   // out-of-range index CRASHES

let t = (dest: "Millbrae", mins: 7)      // tuple
t.mins   t.0   let (d, m) = t            // destructure

var names: [String: String] = ["SBRN": "San Bruno"]
names["SBRN"]           // always String?
names["COLM"] = "Colma" // add/update;  = nil removes
```

- Colon makes a dictionary: `[K: V]` and `["k": v]`. Commas separate items.
- **Trap:** `let c: (Double, Double) = (lon: 1, lat: 2)` drops the labels. Write `(lon: Double, lat: Double)`.
- Dictionary = lookups by key. Fixed named fields = tuple or `struct`.

FUNCTIONS

```
func add(a: Int, b: Int) -> Int { return a + b }
add(a: 2, b: 3)                // call uses labels

func greet(_ name: String)     // call: greet("M")
func greet(to name: String)    // label to, inside name
func fetch(retries: Int = 3)   // default value
func sayHi() { }               // no -> : returns nothing
```

- Return type **must** be written (`-> Type`); Swift doesn't infer it for `func`.
- Labels are part of the name: `data(from:)` and `data(for:)` are different methods.
- Declaration parts: `data(from url: URL)` = label / internal name / type.
- Autocomplete placeholder shows the *type* to supply, e.g. `contentsOfCSVFile: URL`.

ERRORS, ASYNC

```
func load() throws -> Data      // can fail
let d = try load()              // try marks the call
do { try load() } catch { print(error) }  // try/catch
try? load()                     // error becomes nil
func get() async throws -> Data
let d = try await get()         // always try await
```

Plain `try` at top level: an error stops the program ("Error raised at top level").

CLOSURES (ARROW FUNCTIONS)

```
{ a, b in a.time < b.time }    // TS: (a, b) => ...
arr.filter { $0.mins < 10 }     // $0 = first arg
arr.sorted { $0.time < $1.time } // Bool, not -1/0/1
arr.map { $0.name }
arr.first { $0.id == "SBRN" }   // optional
```

Trailing closure: `sorted(by: {…})` → `sorted {…}`. Closures infer return types.

STRUCTS, ENUMS, TYPES AS VALUES

```
struct Station { let id: String; var lat: Double }
let s = Station(id: "SBRN", lat: 37.6)  // init is free
// structs are COPIED on assignment (classes are shared)

enum Dir { case north, south }
switch d { case .north: … case .south: … }
// switch must cover every case

row["stop_id", String.self] // the type, as a value
```

Failable initializer `init?` returns an optional: `URL(string:)` gives `URL?`; `URL(fileURLWithPath:)` gives `URL`.

COMMON ERRORS & WHAT THEY MEAN

Message	Meaning
cannot find 'x' in scope	typo, wrong name, or unquoted text
Type '[X]' has no member…	expected an *array*; wrap in `[ ]`
'guard' body must not fall through	`else` needs return/continue/exit
return invalid outside of a func	top level: use `exit(1)`
initializer for conditional binding must have Optional type	nothing to unwrap; use plain `let`
'nil' cannot initialize type 'X'	make it X?
cannot assign 'URL?' to 'String?'	wrong type annotation
main actor-isolated let… nonisolated context	func reads a top-level var; pass it as a parameter
only available in macOS 13.0 or newer	add `platforms:` to Package.swift
'=' must have consistent whitespace	same spacing both sides of operators

Fix the **first** error on a line; the rest are usually knock-on.

PROJECT: SWIFTPM & PROTOBUF

```
swift package init --type executable \
    --name ArrivalsBoard
swift run --build-system native  // fast rebuilds here
// regenerate the protobuf binding:
protoc --proto_path=Protos --swift_out=Sources \
    Protos/gtfs-realtime.proto
```

```
// Package.swift (argument order matters)
platforms: [.macOS(.v15)],
dependencies: [.package(
  url: "https://github.com/apple/swift-protobuf.git",
  from: "1.39.0")],
.executableTarget(name: "ArrivalsBoard",
  dependencies: [.product(name: "SwiftProtobuf",
                          package: "swift-protobuf")])
```

PROJECT: FETCH, DECODE, GTFS

```
let s = "https://api.bart.gov/gtfsrt/tripupdate.aspx"
guard let url = URL(string: s) else { exit(1) }
let (data, _) =
    try await URLSession.shared.data(from: url)
let feed = try TransitRealtime_FeedMessage(
    serializedBytes: data)
feed.entity.count

let stops = try DataFrame(contentsOfCSVFile:
           URL(fileURLWithPath: "bart-gtfs/stops.txt"))
stops.rows[0]["stop_name", String.self]       // String?
stops.filter(on: "location_type", Int.self) { $0 == 1 }
```

- **Protobuf fields aren't optionals:** missing = default (`0`, `""`). Check `hasTripUpdate`, `hasArrival`.
- `time` is `Int64` (Unix seconds); property is `stopID`; feed `tripID` is a `String`.
- TabularData types: `trip_id` / `location_type` Int, lat/lon Double, `route_id` String. Ask for the right type or get `nil`.
- Top-level vars in main.swift: pass them into functions as parameters.